Guided Proof-Space Evolution: Finite Proof Radius and Completeness-Preserving Resource Allocation for AI-Assisted Mathematics

Alex Chengyu Li
GenAI Architect; Independent Researcher, Tokyo, Japan
cl7076@nyu.edu

August 2026

Abstract

A theorem provable in a declared machine-checkable system has a finite proof code. Exhaustive search therefore has a finite target radius on every positive instance. Yet when the theorem set is undecidable, no computable uniform bound reveals that radius in advance. This tension creates the operational problem for AI-assisted mathematics: within a fixed proof system, guidance cannot change which proof objects are valid, but it can change by exponential factors how much probability and compute reach them.

This paper develops a proof-search control plane for that problem. A complete proposal protocol induces probability mass on candidate proof objects. We define a target's policy-relative distance as the negative logarithm of the total mass assigned to its valid proofs. Under independent verifier-guided sampling, expected candidate count is the inverse of that proof mass. A unit reduction in distance therefore yields a multiplicative search gain. Human judgment, language-model guidance, route selection, and tool choice enter as probability redistribution, not as evidentiary authority.

To prevent guidance from deleting the only valid route, we mix an arbitrary adaptive policy with a support-complete search process. This *completeness reserve* preserves almost-sure discovery of every finite proof and gives an explicit overhead bound; we also prove a deterministic dovetailing analogue. We then lift the model from one target to a portfolio of typed claims and heterogeneous tools. The scheduler values an action by its probability of earning a proof edge, its downstream reachability gain, and its generation, verification, transport, and composition costs. For a homogeneous conjunctive route, an exact one-unit allocation law shows that moving one attempt from an edge with at least two more attempts to the less-funded edge strictly increases closure probability. Candidate artifacts change no mathematical reachability until adequate checking and exact claim binding make their edges admissible.

The resulting control plane is completeness-preserving and verification-cost-aware. Its central object is not an isolated proof tree but the policy that converts heterogeneous search into earned, portfolio-level reachability. The framework composes universal search, neural guidance, claim-bound checking, and proof engineering while preserving their distinct guarantees.

1 Introduction

AI systems can now generate proof sketches, tactic sequences, programs, conjectures, counterexamples, and formal fragments at a rate that changes the organization of mathematical work. Formal

theorem provers increasingly pair a learned proposal policy with a verifier. AlphaProof makes this separation explicit: a policy and value network guide an AND–OR tree search inside Lean; distributed actors receive adaptive compute budgets; and kernel-verified outcomes feed back into training [11]. Recent systems extend the loop to iterative conjecturing and proving [8], natural-language research agents [9], and large-scale formal attacks on open problems [22].

These systems make a resource question unavoidable. Consider a research architecture attacking many mathematical targets in parallel. Each target may admit several routes—sieve, geometric, spectral, probabilistic, or computational—and several tools, including language models, computer algebra, SAT or SMT solvers, specialized domain engines, and proof assistants. Expert judgment may favor one route, but that judgment should change its weight rather than silently erase every alternative. A failed search should lower confidence in the route that failed, not become evidence that the target is false. Likewise, a fast exploratory engine should remain usable without turning its unchecked output into a mathematical premise.

The central problem is therefore not merely to generate more candidates. It is to allocate finite resources over targets, routes, and tools while preserving three different properties:

1. *search coverage*: a finite valid proof should not become permanently unreachable because a learned policy assigns its route zero attention;
2. *evidentiary soundness*: a candidate should change mathematical status only after an adequate, independently checkable procedure has accepted the exact object consumed downstream;
3. *economic composability*: search should account for the cost of verifying, transporting, and composing its outputs, not only for the cost of generating them.

The Build Thesis distinguishes structural enablement from the infrastructure that operationalizes it [17]. Proof Engine Infrastructure supplies a typed claim graph in which only earned evidence-bearing edges derive closure [19]. The claim-level verification framework binds each claim to its object, checker, scope, residual assumptions, and version [16]; the mathematical-engineering account adds dependency closure, resource profiles, and downstream bottlenecks [18]. Taking those components as fixed interfaces, this paper adds the active layer they leave open: how to allocate generation effort before a candidate reaches an evidentiary gate.

The paper makes five contributions.

1. It separates the finite proof radius of a positive instance from the impossibility of a computable uniform radius bound. These elementary results serve as boundary conditions rather than novelty claims.
2. It defines *proof mass* and *policy-relative proof distance* for verifier-guided generation, making the value of guidance an exact multiplicative change in expected search work.
3. It proves a completeness-preservation result for an arbitrary adaptive guided policy mixed with a support-complete baseline, together with a deterministic dovetailing analogue and a finite-portfolio corollary.
4. It defines a proof-search control plane over an earned-claim hypergraph and proves an exact pairwise budget-balancing theorem for homogeneous AND routes, connecting local search response to graph-level closure.
5. It extends the resource objective from generation cost to the total cost of producing independently checkable, semantically bound, and composable proof edges.

Sections 2–4 establish the formal search boundary, policy-relative distance, and completeness reserve. Section 5 lifts these objects to earned-claim hypergraphs and proves the allocation theorem. Section 6 integrates heterogeneous tools and verification boundaries. Sections 7–10 derive system implications, evaluation criteria, prior-work distinctions, and scope limits.

2 The Formal Boundary: Finite Positive Radius, No Uniform Cutoff

We begin with the minimal formal structure needed by the later control model. The claims in this section are classical consequences of effective proof checking, recursive enumerability, and undecidability [4, 7, 23]. Their role is to identify exactly what a search architecture may and may not guarantee.

Definition 2.1 (Machine-checkable proof system)**.** *Let Σ be a finite alphabet. A machine-checkable proof system T is represented here by a total decidable relation*

$$\mathsf{Verify}_T(p, \varphi) \in \{0, 1\}, \qquad p, \varphi \in \Sigma^*.$$

The theorem set is

$$\mathrm{Thm}(T) = \{\varphi : \exists p \, \mathsf{Verify}_T(p, \varphi) = 1\}.$$

The proof code p may encode a linear derivation, a proof tree, a DAG with shared lemmas, a certificate, or any finite artifact accepted by the declared checker.

Definition 2.2 (Proof radius)**.** *For a target φ, define its proof radius relative to T and the chosen encoding by*

$$R_T(\varphi) = \min\{\, |p| : \mathsf{Verify}_T(p, \varphi) = 1 \,\},$$

with $R_T(\varphi) = \infty$ when no such proof code exists.

Theorem 2.3 (Finite positive radius)**.** *If $\varphi \in \mathrm{Thm}(T)$, then $R_T(\varphi) < \infty$. Length-lexicographic enumeration returns a shortest proof after checking at most*

$$\sum_{k=0}^{R_T(\varphi)} |\Sigma|^k = \frac{|\Sigma|^{R_T(\varphi)+1} - 1}{|\Sigma| - 1}$$

candidate strings when $|\Sigma| > 1$.

Proof. Provability supplies at least one finite proof code, so the nonempty set of its lengths has a least element. There are only finitely many strings up to that length. A total verifier checks them one by one, and length ordering makes the first accepted code shortest under the declared encoding. □

Unlike the metaphor of an infinite library containing every finite text, Theorem 2.3 gives a target-conditioned procedure with a decidable acceptance test and a finite certificate at termination. It does not give a feasible algorithm. The bound is exponential in an unknown radius and says nothing about whether the required work fits within physical resources.

Proposition 2.4 (No computable uniform proof-radius bound)**.** *Suppose $\mathrm{Thm}(T)$ is undecidable. There is no total computable function $B : \Sigma^* \to \mathbb{N}$ such that*

$$\varphi \in \mathrm{Thm}(T) \quad \Longrightarrow \quad R_T(\varphi) \le B(\varphi)$$

for every target φ.

Proof. If B existed, compute $B(\varphi)$ and check every proof code of length at most that value. Acceptance would decide that φ is a theorem; failure of all candidates would decide that it is not. This would decide $\mathrm{Thm}(T)$, contradicting the hypothesis. □

The same argument rules out a computable bound depending only on statement length. Every positive instance therefore has a finite target radius, but no general procedure announces a sufficient cutoff before search. Proof length also depends strongly on the chosen formal system, as Gödelian speed-up results show [10]. Proof complexity studies this system-relative efficiency directly [6].

Remark 2.5 (Radius is representation-relative)**.** *A derived lemma can be installed as a reusable named component and then cited in one local step. This does not add a new mathematical consequence, but it can collapse local proof length. A research architecture must therefore distinguish local citation distance from the transitive cost of producing, checking, and maintaining the cited component. The claim graph introduced below charges the latter rather than treating a name as free mathematical progress.*

Three boundaries follow. First, a true but T-unprovable target has infinite radius relative to T; additional compute cannot close it. Second, a failed search does not reveal whether the current radius is enormous or infinite. Third, changing the language, definitions, library, or admissible tools changes the search geometry. Guidance should therefore be evaluated relative to a declared system and resource horizon, not as an intrinsic measure of mathematical truth.

3 Guidance as Probability Redistribution

Exhaustive enumeration treats candidate codes uniformly. LLM-guided theorem proving instead uses a learned proposal distribution. Guidance therefore changes where search probability goes, while the verifier continues to decide which candidates are valid.

Fix a target φ and a protocol for one budgeted attempt. The protocol may run a tactic tree, call tools, or construct auxiliary lemmas. Encode its final finite output as a candidate p, treating timeouts and abandoned attempts as rejecting outcomes. In what follows, π denotes this complete proposal protocol—not the language model alone—and μ_π is its output distribution.

Definition 3.1 (Proof mass and policy-relative distance)**.** *The valid-proof set, proof mass, and policy-relative proof distance are*

$$\begin{aligned} \mathcal{P}_T(\varphi) &= \{p : \mathsf{Verify}_T(p, \varphi) = 1\}, \\ M_\pi(\varphi) &= \sum_{p \in \mathcal{P}_T(\varphi)} \mu_\pi(p), \\ D_\pi(\varphi) &= -\log M_\pi(\varphi), \end{aligned}$$

with $D_\pi(\varphi) = \infty$ when $M_\pi(\varphi) = 0$.

The definition sums over all accepted proof objects, not only a single canonical proof. A policy can therefore improve by concentrating on one good route or by distributing mass across several independent routes.

Proposition 3.2 (Verifier-guided hitting law)**.** *Run independent attempts from a fixed policy π and verify every output. If $M = M_\pi(\varphi) > 0$, then the number N of attempts through the first valid proof is geometric:*

$$\mathbb{E}[N] = \frac{1}{M}, \qquad \mathbb{P}(N > n) = (1 - M)^n \le e^{-nM}.$$

Consequently, $n \ge \log(1/\delta)/M$ attempts suffice for success probability at least $1 - \delta$.

Proof. Each attempt succeeds exactly when its output lies in $\mathcal{P}_T(\varphi)$, an event of probability M. The claims are the standard geometric-distribution identities and the inequality $1 - M \le e^{-M}$. □

Proposition 3.3 (Cost-aware hitting law)**.** *Let (C_j, S_j) be independent, identically distributed pairs of attempt cost and success indicator. Assume $0 < M = \mathbb{P}(S_j = 1)$ and $\mathbb{E}[C_j] < \infty$. Cost may still be correlated with success within an attempt. If N is the first successful attempt, then*

$$\mathbb{E}\left[\sum_{j=1}^{N} C_j\right] = \frac{\mathbb{E}[C_1]}{M}.$$

Proof. The expected total equals the expected successful-attempt cost plus the expected number $(1 - M)/M$ of failures times the expected failure cost:

$$\mathbb{E}[C \mid S = 1] + \frac{1 - M}{M}\mathbb{E}[C \mid S = 0] = \frac{\mathbb{E}[C]}{M}.$$

□

Now suppose a sequential protocol generates a proof path $\rho = (a_1, \ldots, a_L)$ through states $s_1, \ldots, s_L$. Its path probability and path surprisal are

$$\mu_\pi(\rho) = \prod_{j=1}^{L} \pi(a_j \mid s_j), \qquad d_\pi(\rho) = -\sum_{j=1}^{L} \log \pi(a_j \mid s_j).$$

If this path dominates the target's proof mass, the expected attempt count is approximately $e^{d_\pi(\rho)}$. Uniform search with effective branching factor b gives $d = L \log b$, recovering the familiar b^L explosion. Proof length alone is therefore an incomplete search distance. A longer proof with substantial policy mass may be easier to discover than a short proof whose essential construction receives almost none.

Corollary 3.4 (Multiplicative value of guidance)**.** *For two fixed policies with positive proof masses, define the guidance gain from π to π' by*

$$G(\pi \to \pi'; \varphi) = D_\pi(\varphi) - D_{\pi'}(\varphi) = \log \frac{M_{\pi'}(\varphi)}{M_\pi(\varphi)}.$$

Then

$$\frac{\mathbb{E}[N_\pi]}{\mathbb{E}[N_{\pi'}]} = e^G.$$

Measured in bits, $G/\log 2$ is the base-two reduction in effective search work.

This identifies a sharper model-training objective than single-step tactic accuracy. What matters for end-to-end discovery is the total mass retained on at least one complete valid proof. A locally plausible policy can still have infinite target distance if one necessary action receives zero support.

4 Completeness-Preserving Guided Search

Guidance is valuable because uniform enumeration is expensive. Guidance is dangerous because a learned or human-weighted policy can permanently prune the only valid route. The remedy is not to abandon guidance, but to separate a high-throughput guided lane from a small completeness reserve.

Definition 4.1 (Support-complete baseline)**.** *A family of attempt policies $\pi^{\mathrm{f}}_{\varphi}$ is support-complete for T if*

$$\varphi \in \mathrm{Thm}(T) \quad \Longrightarrow \quad M_{\pi^{\mathrm{f}}_{\varphi}}(\varphi) > 0.$$

One construction samples a proof-code length from a full-support distribution, samples uniformly at that length, and invokes Verify_T. A deterministic alternative dovetails the length-lexicographic enumeration of Theorem 2.3.

Let $\pi^{\mathrm{g}}_{t,\varphi}$ be any adaptive guided policy at attempt t. It may depend on human advice, model updates, prior failures, retrieved literature, claim-graph state, and tool feedback. For fixed $\varepsilon \in (0, 1]$, define

$$\pi^{\varepsilon}_{t,\varphi} = (1-\varepsilon)\pi^{\mathrm{g}}_{t,\varphi} + \varepsilon\pi^{\mathrm{f}}_{\varphi}.$$

Theorem 4.2 (Completeness preservation under arbitrary adaptive guidance)**.** *Let $\varphi \in \mathrm{Thm}(T)$, and write $m_{\mathrm{f}} = M_{\pi^{\mathrm{f}}_{\varphi}}(\varphi) > 0$. At each attempt, select the support-complete lane independently with probability $\varepsilon > 0$; otherwise use an arbitrary history-dependent guided lane. Then*

$$\mathbb{P}(\textit{no valid proof in the first } n \textit{ attempts}) \le (1-\varepsilon m_{\mathrm{f}})^n,$$

so a valid proof is found almost surely and

$$\mathbb{E}[N] \le \frac{1}{\varepsilon m_{\mathrm{f}}}.$$

The guided policy may change without restriction and may itself assign zero mass to every valid proof.

Proof. Conditioned on any previous history without success, the next attempt succeeds with probability at least $\varepsilon m_{\mathrm{f}}$ through the baseline lane. Multiplying the conditional failure bounds gives the displayed inequality. The tail tends to zero, and summing it gives the expectation bound. □

Corollary 4.3 (Distance overhead of the completeness reserve)**.** *For a fixed guided policy π^{g},*

$$M_{(1-\varepsilon)\pi^{\mathrm{g}}+\varepsilon\pi^{\mathrm{f}}}(\varphi) \ge \varepsilon M_{\pi^{\mathrm{f}}}(\varphi),$$

hence

$$D_{(1-\varepsilon)\pi^{\mathrm{g}}+\varepsilon\pi^{\mathrm{f}}}(\varphi) \le D_{\pi^{\mathrm{f}}}(\varphi) - \log\varepsilon.$$

The completeness insurance costs at most a factor $1/\varepsilon$ relative to the baseline guarantee and can be much faster when guidance is useful.

The almost-sure conclusion is not a finite physical-time promise. It assumes that attempts continue, that the proof is expressible in the covered system, and that the declared verifier and tools remain available. Its significance is architectural: adaptive weighting need not trade away asymptotic coverage.

Proposition 4.4 (Deterministic dovetailing guarantee)**.** *Let a deterministic baseline enumerator find a proof after N_{f} of its own steps. If a mixed scheduler allocates at least one baseline step in every block of q total steps, then the proof is found within qN_{f} total steps, regardless of all guided work between baseline steps.*

Proof. After qN_{f} total steps the scheduler has executed at least N_{f} baseline steps, including the accepting one. □

Corollary 4.5 (Finite target portfolio)**.** *Consider finitely many active targets $\varphi_1, \ldots, \varphi_n$. Assign each target a positive completeness weight w_i with $\sum_i w_i = 1$, and reserve a positive total fraction ε for a fair scheduler realizing those weights. Every T-provable target receives unbounded cumulative baseline work and is eventually found. Target weights change discovery latency, not the set of asymptotically covered positive instances.*

Together, these results formalize the role of mathematical advice. An expert may assign 60% of guided compute to a sieve route, 25% to a geometric route, and 15% to computation. The agent may update these guided-lane weights as evidence arrives, including setting one temporarily to zero. Completeness is preserved exactly when the total mixed policy retains support through its fair lane; a guided-lane exclusion is therefore a priority decision, not a theorem about impossibility.

5 From a Single Proof Tree to an Earned-Claim Hypergraph

Single-target tactic search is only one part of a research architecture. Research portfolios also contain shared lemmas, alternative routes, conjunctive dependencies, heterogeneous artifacts, and intermediate results whose value lies in the downstream targets they unlock.

Definition 5.1 (Earned-claim graph)**.** *At time t, let*

$$\mathcal{G}_t = (\mathcal{C}, \mathcal{E}_t)$$

be a typed directed hypergraph. Each edge e has a finite input set I_e, an output claim o_e, and an evidentiary state. Given declared roots $R_0 \subseteq \mathcal{C}$, only earned edges propagate reachability:

$$R_{k+1} = R_k \cup \{o_e : e \in \mathcal{E}_t^{\text{earned}},\ I_e \subseteq R_k\}, \qquad \text{Reach}(\mathcal{G}_t) = \bigcup_{k \geq 0} R_k.$$

Multiple incoming edges to one output have OR semantics; multiple inputs of one edge have AND semantics.

This is the minimal portion of Proof Engine Infrastructure needed here [19]. A generated proof sketch, a solver log, or even a locally accepted artifact remains nonpropagating until the exact output claim, assumptions, coverage, and verification boundary satisfy the declared admissibility rule. Search beliefs can move continuously; earned reachability moves only through checked transitions.

Definition 5.2 (Search-control action)**.** *A control action is a tuple*

$$\alpha = (i, e, r, m, b),$$

Here i identifies a target in the active portfolio; e is an open claim edge; r is a mathematical route; m is a generation or analysis tool; and b is the planned verification boundary. The action may yield no progress, edge-relative negative evidence, a candidate artifact, or an artifact that passes the checks required to earn e.

Provability alone is too coarse a scheduling prior. A claim may be highly likely to be provable and still remain unreachable within any relevant budget. Instead, define the budgeted earned-closure distribution

$$F_{t,\alpha}(B) = \mathbb{P}(e \text{ becomes earned within additional budget } B \mid \mathcal{H}_t, \mathcal{G}_t, \alpha),$$

where $\mathcal{H}_t$ records human route assessments, previous attempts, tool behavior, and model state. This distribution combines uncertainty about the route, proof radius, tool suitability, and verification cost. It guides the scheduler; it never replaces evidence.

Let $U(S)$ be a declared utility on reachable claim sets. A simple terminal utility is

$$U(S) = \sum_i u_i \mathbf{1}\{T_i \in S\},$$

but intermediate claims may receive value for reducing a minimal open cut, serving several targets, or creating a reusable theorem component. The realized reachability gain of an earned edge is

$$\Delta U_t(e) = U(\mathrm{Reach}(\mathcal{G}_t \cup \{e\})) - U(\mathrm{Reach}(\mathcal{G}_t)).$$

Before downstream premises are ready this immediate gain may be zero, so a practical scheduler also estimates potential gain through remaining cuts.

Design Principle 5.3 (Closure-aware resource allocation)**.** *Allocate guided resources using the probability that an action produces an earned edge within budget, the edge's expected downstream reachability value, and the total cost of making the output independently consumable. Do not rank actions by candidate-generation rate alone.*

For an additional budget interval ΔB, a heuristic index consistent with this principle is

$$I_t(\alpha; \Delta B) = \frac{\widehat{\Delta U}_t(e) F_{t,\alpha}(\Delta B)}{\mathbb{E}[C_{\mathrm{gen}} + C_{\mathrm{verify}} + C_{\mathrm{transport}} + C_{\mathrm{compose}} \mid \alpha]} + \beta_t \, \mathsf{Explore}_t(\alpha).$$

The exploration term expresses epistemic value and prevents premature route collapse inside the guided lane; the separate completeness reserve protects formal coverage even when the learned index is wrong.

The portfolio-level objective over a total budget B is

$$\max_{\Pi} \mathbb{E}_{\Pi}[U(\mathrm{Reach}(\mathcal{G}_B))] - \lambda \mathbb{E}_{\Pi}[C_{\mathrm{total}}],$$

subject to the completeness allocation and to the rule that only earned edges enter Reach. This is deliberately an objective, not a claim that all terms already admit reliable universal calibration.

The graph exposes allocation effects that a list of independent theorem prompts cannot represent. A difficult lemma on many target cuts may deserve more compute than an easy isolated theorem. Two agents exploring distinct incoming edges may be valuable; two agents unknowingly repeating one route may not be. A failure lowers the posterior of that route and records its obstruction, while other incoming edges to the same claim remain open.

5.1 An exact allocation law for a homogeneous AND route

The general objective does not itself determine an allocation rule. The following tractable case does. Consider two required edges of an AND route. Each independent attempt on either edge fails with the same probability $q \in [0, 1]$. After n attempts, the edge has been earned with probability $1 - q^n$. Independence is explicit here: the model applies to fresh attempts without shared failure causes, not to proof search in general.

Definition 5.4 (Homogeneous edge and two-edge route response)**.** *For $q \in \mathbb{R}$ and $n \in \mathbb{N}$, define*

$$S_q(n) = 1 - q^n.$$

For a nonnegative multiplier R representing the already-funded remainder of the route, define

$$A_q(R; a, b) = R\, S_q(a) S_q(b).$$

When R is the product of the success probabilities of the other required edges, A_q is the closure probability of the whole AND route.

Theorem 5.5 (One-unit balancing identity)**.** *For every $q, R \in \mathbb{R}$ and $a, d \in \mathbb{N}$,*

$$\begin{aligned} &A_q(R; a+1, a+d+1) - A_q(R; a, a+d+2) \\ &\quad = R\, q^a (1-q)(1-q^{d+1}). \end{aligned}$$

Proof. Expand both products. The q^{2a+d+2} terms cancel, and the two remaining differences factor as

$$R(q^a - q^{a+1} - q^{a+d+1} + q^{a+d+2}) = R\, q^a (1-q)(1-q^{d+1}).$$

□

Theorem 5.6 (Strict improvement of an underfunded conjunct)**.** *If $R > 0$, $0 < q < 1$, and one required edge has at least two more attempts than the other, transferring one attempt from the better-funded edge to the underfunded edge strictly increases AND-route closure probability while preserving total search budget. Concretely, for all $a, d \in \mathbb{N}$,*

$$A_q(R; a, a+d+2) < A_q(R; a+1, a+d+1).$$

Proof. Theorem 5.5 gives the gain. Each factor on its right-hand side is strictly positive: $R > 0$, $q^a > 0$, $1 - q > 0$, and $1 - q^{d+1} > 0$ because $d + 1 > 0$ and $0 < q < 1$. □

Corollary 5.7 (Pairwise balance of an optimal homogeneous AND allocation)**.** *Fix $R > 0$ and $0 < q < 1$. If (a, b) maximizes $A_q(R; a, b)$ among all pairs of natural-number budgets with the same total $a + b$, then*

$$a \le b + 1 \qquad \text{and} \qquad b \le a + 1.$$

Equivalently, an optimal pair differs by at most one attempt.

Proof. If, for example, $b \ge a + 2$, write $b = a + d + 2$. Theorem 5.6 constructs the same-total allocation $(a + 1, b - 1)$ with strictly larger closure probability, contradicting optimality. The other inequality is symmetric. □

The theorem is local, but its use is not limited to two-edge problems. In a larger homogeneous AND route, R is the product contributed by all other positively funded required edges. Once those edges have positive success probability, no pairwise-optimal scheduler can give one conjunct at least two more attempts than another. Heterogeneous hazards, shared failure modes, OR routes, and reusable cross-target producers require the fuller control model.

6 Heterogeneous Tools and Verification Boundaries

The control plane must not equate proof search with a single proof assistant. A high-frequency exploratory loop may use a domain-specific geometry engine, a rewriting system, a CAS, a SAT solver, executable search, or an LLM. Lean, Isabelle, Coq, a small certificate checker, or direct witness validation may serve at different cut points. Tool uniformity is not the invariant. The checked object must instead be adequate for the exact claim and composable with its consumers.

For a claim C and verification boundary b, write $V_b(C)$ for the cost of independent checking. For an edge e, write $K(e)$ for the cost of establishing that its checked inputs jointly support its exact output. A target-supporting earned subgraph $\mathcal{G}_T$ has verification-composition cost

$$\sum_{C \in \mathcal{C}(\mathcal{G}_T)} V_{b(C)}(C) + \sum_{e \in \mathcal{E}(\mathcal{G}_T)} K(e),$$

following the graph-level objective in Proof Engine Infrastructure [19]. The present control plane adds the upstream cost of discovering those components and treats boundary choice as part of each search action.

Design Principle 6.1 (Proof-carrying exploration)**.** *Use the fastest suitable engine in the exploratory loop only when its positive outputs can be converted into an independently checkable artifact whose semantics match the claim consumed downstream.*

For example, a specialized geometry engine may derive millions of relations and return a compact construction trace. A narrow checker can validate that trace. Depending on the required assurance, a proof assistant may then verify the checker, the trace, selected semantic interfaces, or the final theorem. There is no need to elaborate every failed branch in a heavyweight environment. Conversely, an expensive search does not make the fast engine's unchecked assertion earned.

The feedback loop is therefore:

human targets, utilities, and route priors
↓
target–edge–route–tool scheduler
↓
parallel candidate generation and analysis
↓
claim-relative verification profiles
↓
candidate-to-earned gate and claim-graph closure
↓
updated blockers, failures, shared lemmas, and resource posteriors
↻

The two planes answer different questions. The control plane asks where to search next; the evidentiary plane asks what the resulting artifact establishes. No probability, model identity, agent reputation, or quantity of compute can substitute for the latter.

7 Implications for LLM Mathematics

7.1 Train for complete-path mass, not only local accuracy

Tactic accuracy, pass@k, and benchmark solve rate remain useful, but they do not explain why a target was missed. A control-plane evaluation should track probability mass along complete

successful paths, identify necessary low-probability actions that were pruned, and measure verified work per unit of total cost. Corollary 3.4 predicts an exponential payoff from modest reductions in cumulative path surprisal.

This interpretation is consistent with current policy/value proof search [11, 24]: the network is valuable not because it certifies a tactic, but because it concentrates finite search on productive branches. The distinction becomes more important at the research frontier, where the relevant path may contain method changes, literature retrieval, new definitions, auxiliary computation, and specialized tools rather than only tactics from an established library.

7.2 Calibrate closure within budget, not timeless confidence

A useful model should estimate a survival curve or hazard for earned closure. That estimate asks how likely an edge is to close in the next unit of compute, given that it has not closed so far. The model should represent proof, refutation, formalization mismatch, and unresolved status separately. In an undecidable setting, a timeout cannot convert "no proof found" into a final unprovability label.

Human guidance enters as a prior over routes and semantic interpretations. Machine experience updates it. Exact refutations, counterexamples, and independence results may justify hard exclusions; ordinary search failure usually justifies a weight change. This is a probabilistic control rule, not a probabilistic definition of mathematical truth.

7.3 Coordinate portfolios through shared cuts

Parallelism is most valuable when agents attack distinct bottlenecks whose outputs compose. The scheduler should expose ownership, route identity, consumer claims, and verification plans before launching work. Shared producer lemmas receive portfolio value; semantically duplicate fragments do not receive artificial credit merely because several agents generated them. Independent replication remains valuable when resources are deliberately allocated to verification rather than accidentally duplicated in discovery.

7.4 Do not optimize generation into a downstream queue

AI-assisted mathematics can make candidate production grow faster than dependency closure, replay, exposition, and maintenance. The resulting bottleneck displacement is developed in the mathematical-engineering account [18]. The control plane incorporates that lesson by pricing verification and composition before allocating search. A route that produces a huge opaque artifact may be mathematically valid yet economically inferior to a route that produces a reusable checked component.

8 Evaluation Program

The framework yields measurements that differ from model-only leaderboards. For each target portfolio and resource horizon, report:

1. earned target closure per GPU-hour, dollar, and verifier-hour;
2. calibration of $F_{t,\alpha}(B)$ against realized earned closure;
3. proof mass or a controlled estimator of complete-path mass, together with realized search work;

4. route diversity and the frequency of necessary actions pruned by the guided lane;
5. reachability gain and reuse count of newly earned edges;
6. generation, verification, transport, and composition cost separately;
7. accumulated verification debt: candidates awaiting an adequate check, semantic binding, or dependency completion;
8. the fraction of closures found through the guided lane and through the completeness reserve.

The following three hypotheses make the evaluation program testable.

Conjecture 8.1 (Guidance-gain calibration)**.** *Holding verifier and attempt protocol fixed, reductions in estimated policy-relative proof distance should predict multiplicative reductions in realized candidate count according to Corollary 3.4, up to dependence, truncation, and cost heterogeneity that must be reported.*

Conjecture 8.2 (Graph-aware portfolio gain)**.** *At equal total compute, scheduling by expected earned reachability gain should close more weighted portfolio targets than scheduling each theorem independently, especially when targets share load-bearing producers.*

Conjecture 8.3 (Engineering complementarity)**.** *The marginal return from stronger LLM guidance should be larger when candidate outputs have predeclared verification boundaries, semantic interfaces, and claim-graph consumers than when proof generation is optimized without those complements.*

These conjectures are falsifiable. Proof-mass estimates may prove too unstable; graph maintenance may cost more than it saves; or specialized tools may create transport debt that dominates their search advantage. The framework requires those costs to be measured rather than assumed away.

9 Relation to Prior Work

Computability and proof length. Recursive enumerability and the negative solution to the general decision problem supply the classical positive-instance/unknown-cutoff boundary [4, 23]. Gödelian speed-up and proof complexity show that proof length depends strongly on the formal system [6, 10]. Dean and Naibo apply computability and complexity limits directly to contemporary AI and mathematical discovery [7]. Section 2 uses these results as operational boundary conditions; it does not claim them as new.

Universal and heuristic search. Levin search allocates resources across candidate programs with a universal length prior. Hutter combines program execution with enumeration of proofs of correctness and runtime bounds [12, 15]. Those systems give broad guarantees for well-defined search problems. The present framework is narrower in universality but broader in research organization: it combines adaptive human/LLM route weights, heterogeneous verification boundaries, multiple mathematical targets, and downstream claim composition. Policy-relative distance serves here as an operational accounting quantity, not as a replacement for algorithmic probability or Kolmogorov complexity.

Portfolio selection and strategy scheduling. Classical ATP systems have long allocated time across heterogeneous strategies rather than relying on one universal configuration. MaLARea couples learning and reasoning over large theorem libraries [13]. Modern Vampire schedules discover and regularize complementary strategies for sequential or parallel execution [2]. More generally, portfolio-based algorithm selection has learning-theoretic guarantees and overfitting limits [1]. These systems directly precede weighted tool and strategy choice. Our control object adds three joint requirements: allocation ranges over typed mathematical claims rather than only prover configurations; success means an earned, semantically bound edge rather than a solver return alone; and utility includes downstream reachability together with verification and composition costs.

Automated theory formation. AM, HR, MATHsAiD, and related theory-exploration systems generate concepts, conjectures, proofs, and counterexamples rather than solving only a supplied theorem [5, 14, 20]. Recent self-play theorem provers continue this direction [8]; Chen and Li model theorems as a semantic-similarity graph and study conditions for exponential self-play growth [3]. Their graph supports curriculum expansion. The graph here instead records typed evidentiary dependency and earned reachability; its control problem is which claim edge to fund and how to verify it.

Neural and agentic theorem proving. Generative proof models, retrieval-augmented provers, and large-scale formal RL systems learn proposal policies and guide tree search [11, 21, 24]. AlphaProof already contains policy/value guidance, progressive sampling, AND nodes, a distributed matchmaker, and adaptive compute budgets. Aletheia iterates generation, verification, and revision in natural language [9]; a recent formal study attacks hundreds of open problems with LLM–Lean agents and reports per-problem costs [22]. These works establish the inner-search precedent. The present contribution is the outer, completeness-preserving and verification-cost-aware control plane over a portfolio of exact claim edges.

Proof Engine and mathematical engineering. The companion architecture separates generation from evidence and derives closure through earned claim edges [19]; the claim-level framework defines heterogeneous verification profiles [16]; and the engineering-layer paper describes dependency closure, component contracts, resource profiles, and downstream bottleneck movement [18]. This paper consumes only their minimal interfaces. It supplies the upstream scheduler and closes the feedback loop from graph state to future search allocation.

10 Limitations and Scope

The results do not imply that every true mathematical statement will be found. They apply to targets with finite proof objects in the covered formal system. They do not select sound new axioms, settle semantic truth beyond the system, or give a finite physical budget for an unknown proof radius.

Policy-relative distance refers to the complete proposal protocol: model policy, search algorithm, decoding parameters, representation, tool set, verifier, and resource cutoff. Changing any of these changes μ_π; the notation does not assert model-only invariance. Exact proof mass is generally unavailable and must be estimated. Correlation between parallel attempts, tree reuse, online learning, and variable verification costs can also make independent-sampling predictions inaccurate. Proposition 3.2 is therefore a baseline law, not a complete performance model.

The scheduler objective contains human choices. Target utilities, acceptable trust boundaries, semantic identity, and the value of explanation are not deduced from proof syntax. Human judgment may itself be wrong, but making its role explicit is preferable to hiding it inside prompts or model scores.

Finally, the completeness reserve is an asymptotic insurance policy. A tiny reserve can be practically irrelevant within a finite project, while a large reserve can consume resources better spent on informed search. Choosing ε is a resource-allocation decision whose empirical optimum will vary by domain. The theorem guarantees what is preserved, not what fraction is economically optimal.

11 Conclusion

A target provable in a declared machine-checkable system has a finite proof radius. For an undecidable theorem set, no computable universal cutoff reveals that radius in advance. AI-assisted mathematics therefore faces finite positive witnesses, potentially enormous and unknowable discovery cost, and no sound timeout-based test for universal failure.

LLMs, human mathematical guidance, and specialized tools address this problem by redistributing probability and compute. Their success is measured by the mass they place on complete valid proof objects and by the cost of turning those objects into earned, composable claims. A completeness reserve allows arbitrary adaptive guidance without permanently deleting finite proofs from the search support. An earned-claim hypergraph then converts local results into a portfolio-level control problem: fund the target–route–tool actions most likely to create valuable verified reachability, while preserving a small fair lane and pricing the downstream verification burden.

This division of labor defines the proposed research method. Humans specify targets, meanings, utilities, and informed route priors. Agents navigate and update the weighted proof space. Specialized engines search at the speed appropriate to their objects, while independent checkers determine evidentiary status. The claim graph records what has actually become reachable. More compute enlarges the practical frontier; guidance changes how quickly it is reached; verification prevents speed from becoming authority.

Declaration of Generative AI Use

Generative AI tools assisted with literature discovery, structural drafting, and copy editing. The author selected the framework, mathematical statements, scope restrictions, and final text, and takes responsibility for the paper.

References

[1] Maria-Florina Balcan, Tuomas Sandholm, and Ellen Vitercik. Generalization in portfolio-based algorithm selection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 12225–12232, 2021. doi: 10.1609/aaai.v35i14.17451.

[2] Filip Bártek, Karel Chvalovský, and Martin Suda. Regularization in spider-style strategy discovery and schedule construction. In *Automated Reasoning – 12th International Joint Conference, IJCAR 2024*, volume 14739 of *Lecture Notes in Computer Science*, pages 223–242. Springer, 2024. doi: 10.1007/978-3-031-63498-7_12. URL `https://arxiv.org/abs/2403.12869`.

[3] Thomas Chen and Zhiyuan Li. A theoretical framework for self-play theorem proving algorithms. arXiv:2606.01861, 2026. URL https://arxiv.org/abs/2606.01861.

[4] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936. doi: 10.2307/2371045.

[5] Simon Colton. *Automated Theory Formation in Pure Mathematics*. Distinguished Dissertations. Springer, London, 2002. ISBN 978-1-85233-609-7. doi: 10.1007/978-1-4471-0147-5.

[6] Stephen A. Cook and Robert A. Reckhow. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, 44(1):36–50, 1979. doi: 10.2307/2273702.

[7] Walter Dean and Alberto Naibo. Artificial intelligence and inherent mathematical difficulty. *Philosophia Mathematica*, 33(3):283–329, 2025. doi: 10.1093/philmat/nkaf005.

[8] Kefan Dong and Tengyu Ma. STP: Self-play LLM theorem provers with iterative conjecturing and proving. arXiv:2502.00212, 2025. URL https://arxiv.org/abs/2502.00212.

[9] Tony Feng, Trieu H. Trinh, Garrett Bingham, et al. Towards autonomous mathematics research. arXiv:2602.10177, 2026. URL https://arxiv.org/abs/2602.10177.

[10] Kurt Gödel. Über die Länge von Beweisen. *Ergebnisse eines mathematischen Kolloquiums*, 7: 23–24, 1936.

[11] Thomas Hubert, Rishi Mehta, Laurent Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09833-y.

[12] Marcus Hutter. The fastest and shortest algorithm for all well-defined problems. *International Journal of Foundations of Computer Science*, 13(3):431–443, 2002. doi: 10.1142/S0129054102001199.

[13] Cezary Kaliszyk, Josef Urban, and Jiří Vyskočil. Machine learner for automated reasoning 0.4 and 0.5. arXiv:1402.2359, 2014. URL https://arxiv.org/abs/1402.2359.

[14] Douglas B. Lenat. AM: An artificial intelligence approach to discovery in mathematics as heuristic search. Technical Report AIM-286; STAN-CS-76-570, Stanford Artificial Intelligence Laboratory, 1976.

[15] Leonid A. Levin. Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.

[16] Alex Chengyu Li. Auditable AI-assisted mathematics: Kernel-checked theorem closure and SAT-guided artifacts. Zenodo, 2026.

[17] Alex Chengyu Li. The build thesis: Why mathematics's AI acceleration is not reducible to structural conditions, 2026. Draft manuscript.

[18] Alex Chengyu Li. Beyond kernel verification: The missing engineering layer of mathematics. SSRN, 2026.

[19] Alex Chengyu Li. Proof engine infrastructure: A claim-graph framework for AI-assisted mathematical research. Zenodo, 2026.

[20] Roy L. McCasland, Alan Bundy, and Patrick F. Smith. MATHsAiD: Automated mathematical theory exploration. *Applied Intelligence*, 47:585–606, 2017. doi: 10.1007/s10489-017-0954-8.

[21] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020. URL `https://arxiv.org/abs/2009.03393`.

[22] George Tsoukalas, Anton Kovsharov, Sergey Shirobokov, et al. Advancing mathematics research with AI-driven formal proof search. arXiv:2605.22763, 2026. URL `https://arxiv.org/abs/2605.22763`.

[23] Alan M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1937. doi: 10.1112/plms/s2-42.1.230.

[24] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.52202/075280-0944. URL `https://arxiv.org/abs/2306.15626`.